

CSSE 230 -- Data Structures and Algorithms

Final Exam Spring 2018 Part 2 Name: _____ **01 02 03**
(and 1 extra problem from another exam)

Rules for this part:

You may use any printed or written materials that you brought with you, code that you wrote yourself or with a partner before the exam, anything on the CSSE230 Moodle and web sites, and the Java API documentation (on your laptop or at oracle.com).

You may not otherwise search the internet or communicate with any person other than your instructor or a student assistant. You may not communicate with anyone else about this exam until allowed by your instructor.

You will sign a statement to certify that you neither received help from others nor gave it on either part of the exam.

You must actually get these problems working on your computer. Most of the credit for the problems will be for code that actually works, and the rest is for efficiency. Little or no credit will be given for “good ideas.”

You generally shouldn’t have to make any changes to the given code other than to implement the required methods and perhaps to add helper methods as needed.

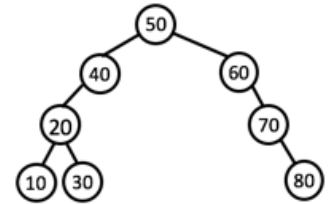
Instructor use only:

Problem	Possible	Score	Grader Comments
1	20		____/10 unit tests + ____/10 efficiency
2	20		
3	20		
Total	60		

Check out the FinalExam project from your csse230 SVN repository.

1. (20 points) **BinaryTree with efficiency.**

In the `tree` package, write `int countExactlyBalancedNodes()`, that returns the number of nodes in the given tree whose subtrees have the same height. In other words, while this is not an AVL tree (so does not need to be height-balanced), if it were, this method would count the number of nodes with balance code of SAME. This method would return 5 for the tree to the right (20, 50, and the 3 leaves)



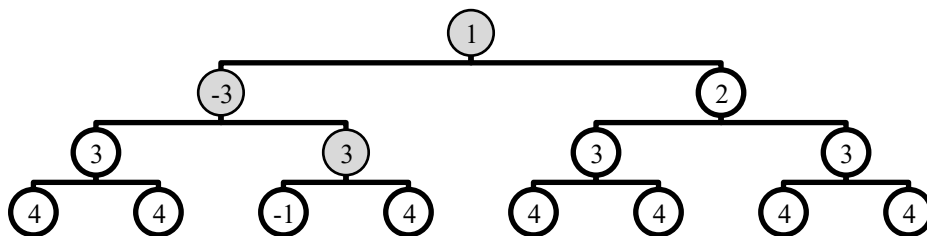
Efficiency. 10 points will be awarded for passing the JUnit tests, and 10 for efficiency. To earn the efficiency points, you must

- (a) Earn all 10 points for correctness
- (b) Not add any fields to any of the existing classes (you can add a new class if you wish)
- (c) Have your code run in $O(N)$ time
- (d) Not have your code descend into any subtree more than once.

2. (20 points) **minIndexOfHeapRoot** for an array of ints, interpreted as having heap structure.

In the `heap` package, implement the static method `minIndexOfHeapRoot(int[] arr)`: interpreting the given array using standard 1-indexed “heap structure,” return the minimum index x such that, in the tree interpretation of the heap, x is the root of a valid min-heap. That is, x is the minimum index whose descendants all satisfy the heap-order property. Note that such an index will always exist, as long as the given array has size at least 2, because the leaf nodes are trivially heaps. (You don’t have to deal with the case of array size ≤ 1 ; i.e. you don’t need to worry about empty heaps.)

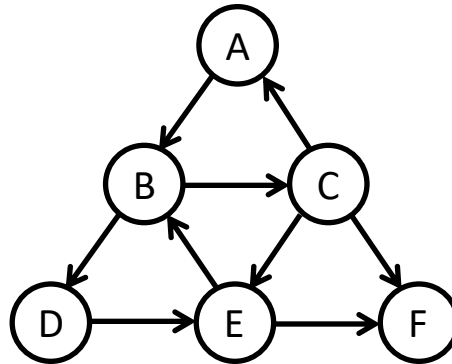
For instance, the array $\{0, 1, -3, 2, 3, 3, 3, 3, 4, 4, -1, 4, 4, 4, 4, 4\}$ corresponds to the following tree, where all except the three greyed-out nodes are roots of valid min-heaps. Among those nodes, the one of minimum index is the one in index 3 (of value 2), so the method should return 3.



3. (20 points) **numDirectedTriangles** for the Graph implementations.

In the `graph` package, you are given an abstract `Graph` class along with adjacency-list and adjacency-matrix implementations. (The implementations have some of the `GraphSurfing` Milestone 1 methods implemented; feel free to add any more you think are helpful.)

Implement the **`int numDirectedTriangles()`** method, which should return the number of directed triangles (3-cycles) in the graph. A directed triangle is a set of 3 nodes, say A, B, and C, having edges $A \rightarrow B \rightarrow C \rightarrow A$. For instance, the following graph has three directed triangles, namely ABC, BCE, and BDE.



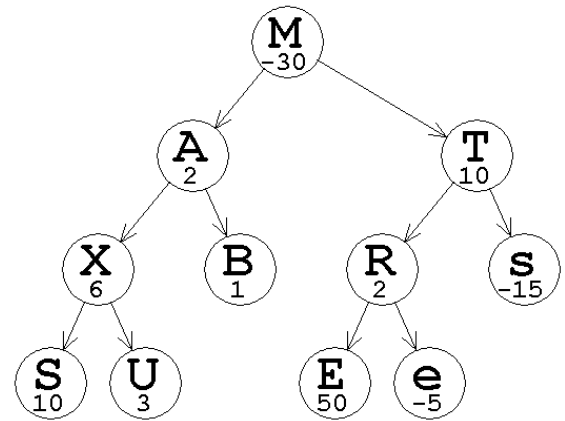
You may implement this method in one of two ways:

- Implement it in both `AdjacencyList` and `AdjacencyMatrix` `Graph` implementations
- Implement it in the `Graph` class, removing the “abstract” modifier from the method declaration

The latter should be easier if you can complete it using only methods in the `Graph` class.

One of the unit tests (worth 2 points in each implementation) contains an absolute time constraint for computing **`numDirectedTriangles()`** on a large dense graph. To assure you will be awarded these points, your code should consistently and comfortably pass this test.

4. (30 points) Another even harder binary tree with efficiency, that came from a different exam (we typically only have 3 problems on the final exam): **maxSubtree**. In the **subtree** package, implement **String maxSubtree()**. We'll consider trees with two "data" elements: a label and a value – see the given example. This method finds the subtree in the given tree whose sum has the maximum sum and returns a string containing the label of its root and the sum of the values in the subtree. If more than one maximum subtree has the same sum, return the left-most one. If the tree is empty or the sum is negative, return **0** for the sum and **\$** as the label to signify that the empty tree has the max sum (much like when the MCSS of a list with all negatives was 0 for the empty list).



For example, in the tree to the right (from the unit tests), the subtree at A has sum 22 and the subtree at R has sum 47. The method would return E50 for the max, but if the value of E were only 22 or less, the method would return A22.

You may earn up to 20 points for any method that passes the unit tests (and is general enough to pass other tests on larger trees). The other 10 points are for your code also having efficiency of $O(n)$ where the tree has n nodes, therefore the code may only visit each node once. As usual, you may not include any other fields in the **BinaryTree** class, but you may add helper methods with parameters and return values of any type, including a class that you define.